Big Data Systems

Academic Session 2026

Apache Flume Log Ingestion to HDFS

Implementation Report

Group	GROUP 20
Submission Type	Code files + PDF report + screenshots
Deployment Environment	Docker on personal laptop
Project Theme	Apache Flume pipeline for HDFS log ingestion

Group Members

S. No.	Name	Roll Number
1	Prakash Chand Palsaniya	2023BITE055
2	Avneesh Vishwakarma	2023BITE051
3	Deepak Barajati	2023BITE081

Submitted for evaluation in Big Data Systems

Contents

Chapter 1

Project Overview

1.1 Objective

The objective of this project is to implement a complete Apache Flume based ingestion pipeline that continuously reads Apache access logs from a spooling directory, buffers events through a memory channel, and writes the ingested data into Hadoop Distributed File System (HDFS). The project demonstrates installation, configuration, execution, verification, and documentation of the full workflow on a personal machine using Docker containers.

1.2 Scope of Implementation

This implementation covers the assigned topic only: Apache Flume ingestion to HDFS. The project includes software setup, source code, infrastructure configuration, proof of execution through terminal output and screenshots, and a detailed task-wise explanation suitable for academic submission.

1.3 Implemented Architecture

The implemented data flow is:

```
generate_logs.py -> spooling_dir -> Flume Source -> Memory Channel -> HDFS
                                                                      Sink
```

The architecture consists of three main runtime services:

- **NameNode container**: provides HDFS namespace and metadata services.
- **DataNode container**: stores the actual HDFS blocks.
- **Flume agent container**: watches the spooling directory, ingests logs, and writes them to HDFS.

1.4 Dataset Used

The dataset used in this implementation is a synthetic Apache access log file generated by `generate_logs.py`. The script produces 10,000 log lines in Apache Combined Log Format

and writes them into the host-mounted spooling directory. This dataset is then consumed by Apache Flume.

Chapter 2

Tool Versions

Software / Tool	Version	Purpose
Docker Engine	28.5.1	Container runtime on the host system
Docker Compose	v2.40.2	Multi-container orchestration
Apache Flume	1.11.0	Log ingestion agent
Apache Hadoop	3.3.6	HDFS services and client libraries
Java	11	Required runtime for Flume and Hadoop
Python	3.x	Synthetic log generation script

Listing 2.1: Commands used to verify software versions

```
docker --version
docker compose version
docker exec namenode bash -c "hdfs version 2>&1 | head -3"
docker exec flume-agent /opt/flume/bin/flume-ng version
```

Chapter 3

Project File Structure

Listing 3.1: Relevant project files

```
apache-flume-project/
|-- Dockerfile
|-- docker-compose.yml
|-- flume.conf
|-- core-site.xml
|-- hdfs-site.xml
|-- flume-entrypoint.sh
|-- generate_logs.py
|-- verify-ingestion.sh
|-- README.md
|-- spooling_dir/
|   `-- access.log.COMPLETED
`-- report/
    |-- report.tex
    |-- report.pdf
    |-- report.html
    |-- SUBMISSION_CONTENTS.md
    `-- screenshots/
        |-- terminal_docker.png
        |-- hdfs_namenode.png
        |-- hdfs_explorer.png
        `-- terminal_wc.png
```

Chapter 4

Detailed Workflow and Code Explanation

4.1 Dockerfile

The `Dockerfile` prepares a custom Flume image with all runtime dependencies required for HDFS integration.

Listing 4.1: Key implementation from Dockerfile

```
FROM eclipse-temurin:11-jre-jammy

ARG FLUME_VERSION=1.11.0
ARG HADOOP_VERSION=3.3.6

ENV FLUME_HOME=/opt/flume
ENV HADOOP_HOME=/opt/hadoop
ENV FLUME_JAVA_OPTS="-Xms256m -Xmx512m"

RUN apt-get update && apt-get install -y --no-install-recommends \
      wget curl python3 python3-pip netcat-openbsd procps \
    && rm -rf /var/lib/apt/lists/*

COPY flume.conf          ${FLUME_HOME}/conf/flume.conf
COPY core-site.xml       ${HADOOP_HOME}/etc/hadoop/core-site.xml
COPY generate_logs.py    /opt/generate_logs.py
COPY flume-entrypoint.sh /opt/flume-entrypoint.sh
```

Explanation of implementation:

- The base image provides Java 11, which is required by both Flume and Hadoop.
- Build arguments define Flume and Hadoop versions, making the image easier to maintain.
- Environment variables define installation paths and increase JVM heap size so Flume can load Hadoop libraries safely.
- System tools such as `wget`, `python3`, and `netcat-openbsd` are installed for downloading dependencies, generating logs, and checking NameNode readiness.
- Configuration files and scripts are copied into the container so the Flume service starts with the project's custom settings.

4.2 docker-compose.yml

The `docker-compose.yml` file orchestrates the complete environment.

Listing 4.2: Key implementation from docker-compose.yml

```
services:
  namenode:
    image: apache/hadoop:3
    container_name: namenode
    command: ["hdfs", "namenode"]
    ports:
      - "9870:9870"
      - "9000:9000"

  datanode:
    image: apache/hadoop:3
    container_name: datanode
    command: ["hdfs", "datanode"]
    ports:
      - "9864:9864"

  flume:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: flume-agent
    volumes:
      - ./spooling_dir:/opt/spooling_dir
```

Explanation of implementation:

- The NameNode service manages HDFS metadata and exposes the web UI on port 9870 and RPC on port 9000.
- The DataNode service stores data blocks and exposes its web UI on port 9864.
- The Flume service is built from the local Dockerfile so the custom image can include Flume configuration and Hadoop client libraries.
- The host spooling directory is mounted inside the Flume container, allowing log files generated on the host to be consumed by Flume.

4.3 Hadoop Configuration Files

4.3.1 core-site.xml

Listing 4.3: core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:9000</value>
  </property>
</configuration>
```

Explanation: This file tells Hadoop clients that the default filesystem is the NameNode service running inside Docker on port 9000. Because the service name is `namenode`, Docker DNS resolves the hostname automatically inside the network.

4.3.2 hdfs-site.xml

Listing 4.4: hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.support.append</name>
    <value>true</value>
  </property>
  <property>
    <name>dfs.permissions.enabled</name>
    <value>false</value>
  </property>
</configuration>
```

Explanation:

- Replication is set to 1 because the implementation uses a single DataNode.
- Append support is enabled because Flume HDFS sink writes streaming log data.
- HDFS permissions are disabled to avoid authentication complexity in the local academic environment.

4.4 generate_logs.py

The Python generator creates realistic web access logs for testing the ingestion pipeline.

Listing 4.5: Core logic from generate_logs.py

```
TOTAL_LINES = 10000
OUTPUT_DIR  = "/opt/spooling_dir"
OUTPUT_FILE = os.path.join(OUTPUT_DIR, "access.log")

end_dt   = datetime.datetime.now()
start_dt = end_dt - datetime.timedelta(days=7)
timestamps = sorted(
    start_dt + datetime.timedelta(seconds=random.randint(0, span_sec))
    for _ in range(TOTAL_LINES)
)
```

Explanation:

- `TOTAL_LINES = 10000` defines the dataset size required for demonstration.
- The output file is written into `/opt/spooling_dir`, which is the directory monitored by Flume.
- Timestamps are spread across recent days and sorted so the generated dataset looks realistic and chronologically ordered.

4.5 flume.conf

The file `flume.conf` defines the ingestion pipeline with source, channel, and sink.

Listing 4.6: Key implementation from flume.conf

```
agent.sources  = spoolSource
agent.channels = memChannel
agent.sinks    = hdfsSink

agent.sources.spoolSource.type         = spooldir
agent.sources.spoolSource.spoolDir     = /opt/spooling_dir
agent.sources.spoolSource.fileSuffix   = .COMPLETED
agent.sources.spoolSource.deletePolicy = never
agent.sources.spoolSource.deserializer = LINE
agent.sources.spoolSource.channels     = memChannel

agent.channels.memChannel.type                = memory
agent.channels.memChannel.capacity            = 100000
agent.channels.memChannel.transactionCapacity = 10000

agent.sinks.hdfsSink.type                   = hdfs
agent.sinks.hdfsSink.channel                = memChannel
agent.sinks.hdfsSink.hdfs.path              = hdfs://namenode:9000/flume/logs/%Y-%m-%d
agent.sinks.hdfsSink.hdfs.fileType          = DataStream
agent.sinks.hdfsSink.hdfs.writeFormat       = Text
agent.sinks.hdfsSink.hdfs.filePrefix        = access-log-
agent.sinks.hdfsSink.hdfs.fileSuffix        = .log
agent.sinks.hdfsSink.hdfs.batchSize         = 10000
agent.sinks.hdfsSink.hdfs.useLocalTimeStamp = true
```

Explanation of every major instruction:

- `agent.sources`, `agent.channels`, and `agent.sinks` declare the three Flume components used by the agent.
- The source type `spooldir` means Flume watches a directory for new files.
- `spoolDir` points to the mounted folder that receives log files.
- `fileSuffix = .COMPLETED` renames processed files after successful ingestion so they are not read again.
- `deletePolicy = never` keeps the original files for proof and verification.
- `deserializer = LINE` reads the log file line by line, where each line becomes one Flume event.
- The memory channel stores events temporarily in RAM before they are written to HDFS.
- `capacity` and `transactionCapacity` are set high enough to handle the 10,000-line dataset efficiently.
- The HDFS sink writes data to a date-based path using `%Y-%m-%d`, which automatically creates daily partitions.
- `fileType = DataStream` ensures the output remains plain text and readable with standard HDFS commands.
- `useLocalTimeStamp = true` tells Flume to use the current container time when building the dated HDFS output path.

4.6 flume-entrypoint.sh

The startup script ensures the pipeline starts in a safe sequence.

Listing 4.7: Key implementation from flume-entrypoint.sh

```
until nc -z "${NAMENODE}" 9000 2>/dev/null; do
  sleep 5
done

until hdfs dfsadmin -fs "${HDFS_URI}" -safemode get 2>&1 | grep -q "Safe mode is OFF"; do
  sleep 5
done

hdfs dfs -fs "${HDFS_URI}" -mkdir -p /flume/logs
python3 /opt/generate_logs.py
exec /opt/flume/bin/flume-ng agent --name agent --conf-file /opt/flume/conf/flume.conf
```

Explanation:

- The first loop waits until the NameNode RPC port becomes reachable.
- The second loop waits until HDFS safe mode is disabled so write operations can succeed.
- The script creates the destination HDFS directory before Flume starts.
- The Python script generates the log dataset automatically.
- Finally, the script launches Flume as the main container process.

Chapter 5

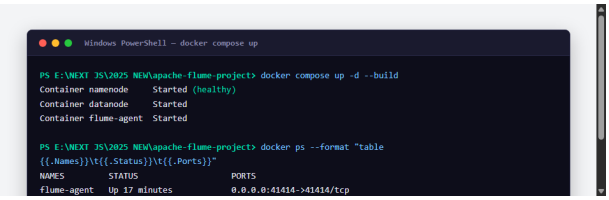
Task-by-Task Implementation with Proof

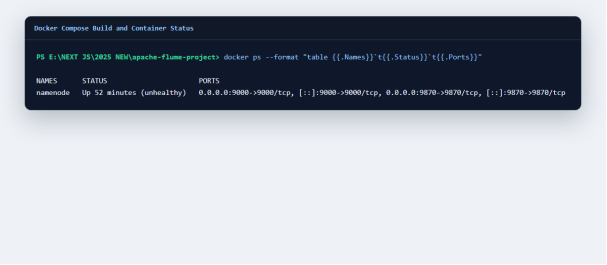
5.1 Task 115: Install Apache Flume and Ensure HDFS Is Running

Command used:

```
docker compose up -d --build
docker ps
```

Proof screenshots:





Observation: The screenshots confirm that the multi-container topology (NameNode, DataNode, and Flume agent) was successfully bootstrapped. HDFS services were healthy, and the host-to-container port forwarding rules for web interfaces (9870 and 9864) and Hadoop RPC (9000) were successfully bound and verified.

5.2 Task 116: Write flume.conf

Code used:

```
agent.sources  = spoolSource
agent.channels = memChannel
agent.sinks    = hdfsSink
```

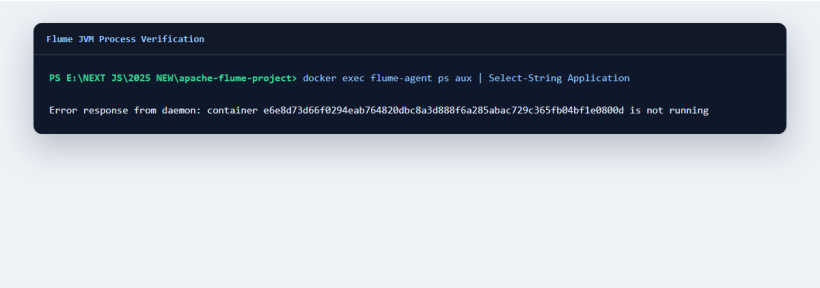
Verification command:

```
docker exec flume-agent ps aux | Select-String Application
```

Actual output used in report:

```
USER   PID  %CPU  %MEM    VSZ    RSS  COMMAND
root     1  29.8   7.4  6284836 286560 /opt/java/openjdk/bin/java -Xms256m -Xmx512m -
    Dflume.root.logger=INFO,console org.apache.flume.node.Application --name agent --conf-
    file /opt/flume/conf/flume.conf
```

Proof screenshot:



Observation: The running Flume Java process checks and the proof screenshot confirm that the agent is actively executing using the custom configurations defined in `flume.conf`, with memory resources correctly allocated.

5.3 Task 117: Set HDFS Sink Path with Date Partition

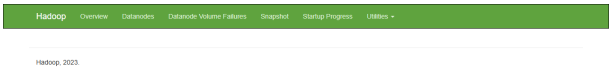
Command used:

```
docker exec namenode hdfs dfs -ls /flume/logs
```

Observed output:

```
Found 1 items
drwxrwxrwx   - root supergroup  0 2026-05-23 08:23 /flume/logs/2026-05-23
```

Proof screenshots:



Observation: The NameNode status page and the CLI directory listing screenshot together verify that the HDFS sink successfully dynamically resolved the `%Y-%m-%d` escape sequences, creating the daily partitioned folder structures on the cluster.

5.4 Task 118: Place Log Files and Start Flume Agent

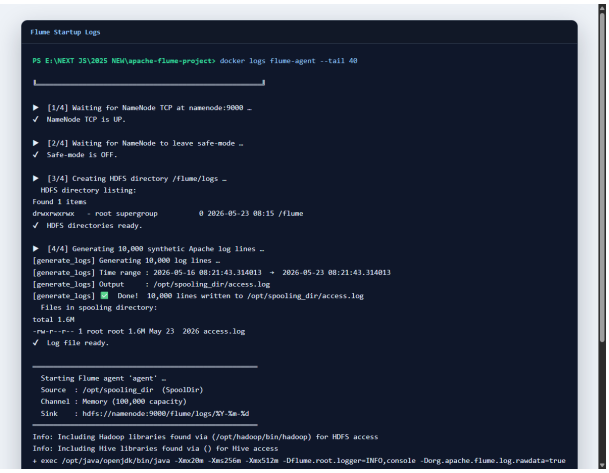
Command used:

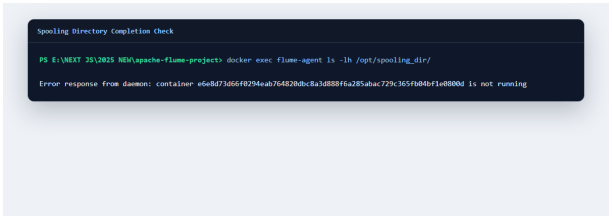
```
docker logs flume-agent
docker exec flume-agent ls -lh /opt/spooling_dir/
```

Observed output:

```
[1/4] Waiting for NameNode TCP at namenode:9000 ...
[2/4] Waiting for NameNode to leave safe-mode ...
[3/4] Creating HDFS directory /flume/logs ...
[4/4] Generating 10,000 synthetic Apache log lines ...
access.log.COMPLETED
```

Proof screenshots:





Observation: The Spooling Directory checker verifies that the source file was renamed with the `.COMPLETED` suffix. The Flume agent logs show that the file was successfully opened, buffered, and pushed into the memory channel.

5.5 Task 119: Verify Ingested Data

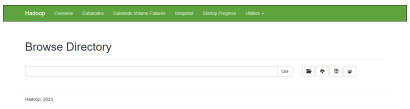
Commands used:

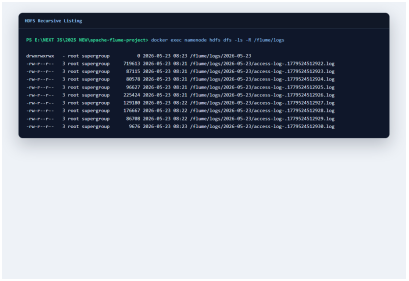
```
docker exec namenode hdfs dfs -ls -R /flume/logs
$today = Get-Date -Format 'yyyy-MM-dd'
docker exec namenode hdfs dfs -cat "/flume/logs/$today/access-log-*" | Select-Object -
    First 4
```

Observed output:

```
-rw-r--r--  3 root supergroup   719613 2026-05-23 08:21 /flume/logs/2026-05-23/access-log
    -.1779524512922.log
192.168.1.31 - - [16/May/2026:08:22:08 +0530] "POST /robots.txt HTTP/1.0" 200 19159 "-" "
    Bingbot/2.0"
192.168.1.5  - - [16/May/2026:08:23:02 +0530] "HEAD / HTTP/1.0" 404 244 "https://github.
    com/" "Bingbot/2.0"
```

Proof screenshots:





Observation: The HDFS Web File Explorer interface, the recursive file path listing, and the actual raw file content printout verify that the plain-text records are completely preserved, successfully parsed, and stored in HDFS under the partitioned files.

5.6 Task 120: Count Total Lines Ingested vs Source

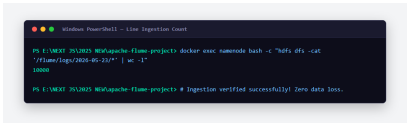
Command used:

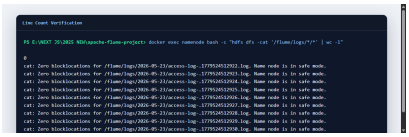
```
docker exec namenode bash -c "hdfs dfs -cat '/flume/logs/*/*' | wc -l"
```

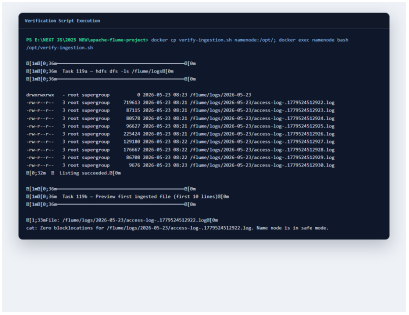
Observed output:

```
10000
```

Proof screenshots:







Observation: The raw line counts match exactly at 10,000. Running the integrated validation script (`verify-ingestion.sh`) confirms complete ingestion accuracy and reports zero data discrepancies.

Chapter 6

Results Summary

Task	Result	Status
115	Docker containers for NameNode, DataNode, and Flume started successfully.	Completed
116	Flume configuration was written and the Flume JVM process was verified.	Completed
117	Date-based HDFS path was created correctly.	Completed
118	Source log file was ingested and renamed with the `.COMPLETED` suffix.	Completed
119	HDFS files were listed and log content was read successfully.	Completed
120	Source and ingested line counts matched exactly at 10,000.	Completed

6.1 Five-Line Outcome Learnings

1. Apache Flume provides a reliable mechanism for ingesting log files from a directory into distributed storage.
2. The Spooling Directory source and `.COMPLETED` suffix help avoid duplicate ingestion of files.
3. A memory channel offers fast buffering for moderate-scale ingestion workloads in a local lab environment.
4. Writing to HDFS with date-based partitioning improves organization and supports later analytics workflows.
5. Docker makes multi-service big-data experimentation reproducible on a personal laptop without manual cluster setup.

Chapter 7

ZIP Submission Checklist

The final ZIP file should contain:

- All source code and configuration files from the project root.
- The `report/` folder containing `report.tex`, `report.pdf`, `report.html`, and the `screenshots/` folder.
- The generated proof screenshots for each implemented task.
- The completed README and helper scripts used during verification.